MAANG Interview Guide: JS Iterators & Loops

Comprehensive Q&A covering iterators, generators, and asynchronous execution semantics.

Section 1: Iterators and Iterable Protocol

1. What is the difference between an iterable and an iterator?

> An **iterable** is an object that implements the `Symbol.iterator` method, which returns an iterator. An **iterator** is an object that implements the `next()` method, returning an object with `value` (the next item) and `done` (a boolean indicating if the sequence is finished).

2. Why does for...of work with arrays, but not plain objects?

> Arrays, Strings, Maps, and Sets have built-in `Symbol.iterator` properties. Plain objects do not implement this protocol by default because object property iteration order isn't strictly defined in a single logical sequence, and `for...in` is historically used for object keys.

3. Implement a custom iterable between a start and end value.

```
const range = (start, end) => ({
  [Symbol.iterator]: () => {
    let current = start;
    return { next: () => (current <= end ? { value: current++, done: false } : { done: true
}) };
  }
});
```

4. { value: undefined, done: false } vs { value: 100, done: true }?

> The first yields the actual value `undefined` while continuing the sequence. The second signifies the iterator is complete, but returns a final "return" value of `100` (often ignored by `for...of` but accessible if `next()` is called manually).

5. What is iterator closing and return()?

> Iterator closing happens when a loop terminates early (via `break`, `return`, or a thrown error). `for...of` automatically calls the iterator's optional `return()` method to allow resource cleanup (e.g., closing file handles or database connections).

Section 2: Generators and Delegation (Part 1)

1. What is a generator function? Explain function*, yield, and next().

> A generator (`function*`) is a function that can pause its execution and resume later. `yield` pauses execution and emits a value. Calling `next()` resumes execution until the next `yield`, returning an iterator result object.

2. Why is the argument to the first next(value) ignored?

> The first `next()` starts the generator, executing code up to the first `yield`. There is no preceding `yield` expression evaluating to receive the passed value. Subsequent `next(val)` calls substitute `val` into the previously paused `yield`.

Section 2: Generators and Delegation (Part 2)

3. Difference between yield and yield*?

> `yield` pauses and returns a single value. `yield*` delegates iteration to another iterable or generator, yielding all its values one by one. The return value of the delegated iterator becomes the result of the `yield*` expression.

4. What happens when return() or throw() is called on a suspended generator?

> `return(val)` forces the generator to complete (done: true) and returns `val`, executing any active `finally` blocks. `throw(err)` injects an exception at the suspended `yield`. If wrapped in `try...catch` inside the generator, the generator catches it and can continue; otherwise, it terminates.

5. How do generators implement lazy evaluation?

> Generators only compute the next value when `next()` is called. This prevents loading large datasets into memory at once, allowing infinite sequences (like an endless Fibonacci generator) with O(1) memory complexity instead of O(N).

Section 3: Async Iterators and Async Generators

1. Symbol.iterator vs Symbol.asyncIterator?

> `Symbol.iterator` returns an object whose `next()` returns `{ value, done }`. `Symbol.asyncIterator` returns an object whose `next()` returns a **Promise** resolving to `{ value, done }`, designed for asynchronous streams.

2. Behavior of for await...of?

> It consumes async iterables, awaiting each Promise before proceeding to the next iteration. If given a synchronous iterable yielding Promises, it implicitly awaits each yielded Promise sequentially.

3. for...of + await vs Promise.all()?

> `for...of` with `await` executes operations strictly **sequentially**; the next loop iteration waits for the previous to finish. `Promise.all()` executes operations **concurrently** in parallel, waiting only for all to resolve before moving forward.

4. Implement an async generator for paginated APIs.

> ```
> async function* fetchPages(url) {
>   let page = 1;
>   while (true) {
>     const res = await fetch(`${url}?page=${page++}`);
>     const data = await res.json();
>     if (!data.length) break;
>     yield* data; // yield each record
>   }
> }
> ```

Section 4: Traditional Loops and Control Flow

1. Execution order of a for loop?

> 1) Initialization (once). 2) Condition check (if false, exit). 3) Body execution. 4) Update expression. 5) Repeat step 2. Omitted clauses default to true (condition) or no-op (init/update).

2. Compare for, while, and do...while.

> `for`: best for known bounds/index iteration. `while`: best when termination depends on an external/dynamic condition evaluated before execution. `do...while`: guarantees the body executes at least once before checking the condition.

3. break, continue, and return?

> `break` exits the innermost loop entirely. `continue` skips the rest of the current iteration and jumps to the update/condition phase. `return` exits the enclosing function entirely, terminating all loops inside it immediately.

4. Labeled break and continue?

> Labels (e.g., `outer: for(...)`) allow breaking or continuing a specific outer loop from within a nested loop (`break outer;`). It improves control flow in deep nesting but can harm readability, resembling goto statements.

5. How can continue create an infinite loop in a while loop?

> In a `for` loop, `continue` jumps to the update expression (e.g., `i++`). In a `while` loop, `continue` jumps straight to the condition check. If the incrementer is bypassed by the `continue`, the condition never changes, causing an infinite loop.

Section 5: for...of, for...in, and Object Iteration (Part 1)

1. Compare enumeration methods.

> `for...of`: iterable values. `for...in`: enumerable string keys (including prototype chain). `Object.keys/values/entries`: own enumerable string keys/values/pairs. `Reflect.ownKeys`: all own keys (including non-enumerable and Symbols).

2. Why avoid for...in for arrays?

> `for...in` iterates over keys (indices as strings), not values. It also iterates over inherited enumerable properties on the Array prototype. Order is not strictly guaranteed in older engines, making it risky for sequential array access.

Section 5 & 6: Array Iteration

3. Iterating arrays with holes (sparse arrays)?

> Index-based loops (`for (let i=0...)`) and `for...of` visit the holes, treating them as `undefined`. Array methods like `forEach`, `map`, and `for...in` skip the holes entirely.

4. Iterating strings with Unicode surrogate pairs?

> Index-based iteration splits surrogate pairs (emojis/special chars) into two garbage characters because it iterates by 16-bit code units. `for...of` uses the string's iterator, which correctly yields full Unicode code points.

5. Array mutation during for...of?

> The iterator uses the array's current state. Appending elements will cause them to be visited. Deleting upcoming elements will cause them to be skipped. Shifting elements shifts the index tracker, causing elements to be skipped.

S6 Q1. Compare forEach, map, filter, reduce.

> `forEach`: executes callback, returns undefined. `map`: transforms elements, returns new array. `filter`: returns new array of elements passing a test. `reduce`: accumulates values into a single return value. None mutate the original array by default.

S6 Q2. Short-circuiting methods?

> `some()` stops returning true on the first truthy result; `every()` stops returning false on the first falsy result. `find/findIndex` stop on the first match. On empty arrays: `some` returns false, `every` returns true.

S6 Q3. forEach with async callbacks?

> `forEach` fires the callbacks synchronously and does not wait for Promises to resolve. To process sequentially, use `for...of` with `await`. To process concurrently, map to an array of Promises and use `Promise.all()`.

S6 Q5. reduce() on empty arrays?

> If no initial value is provided, calling `reduce()` on an empty array throws a `TypeError`. Providing an initial value safely returns that initial value.

Section 7: Output-Based Interview Questions

31. What is the output of the following code, and why?

```
for (let i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 0);
}
```

> **Output: 0, 1, 2.** `let` is block-scoped. Each iteration creates a new binding for `i`, which the closure inside `setTimeout` captures perfectly.

32. What is the output, and why?

```
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 0);
}
```

> **Output: 3, 3, 3.** `var` is function-scoped. There is only one `i` variable. By the time the event loop executes the timeouts, the synchronous loop has finished, and `i` is 3.

33. What is the output, and why does the generator behave this way?

```
function* generate() {
  console.log("A");
  yield 1;
  console.log("B");
  return 2;
}

const iterator = generate();
console.log("C");
console.log(iterator.next());
console.log(iterator.next());
console.log(iterator.next());
```

> **Output:** C, A, {value: 1, done: false}, B, {value: 2, done: true}, {value: undefined, done: true}. The generator doesn't execute its body until `next()` is called, so 'C' logs first.

34. What is the output, and why is the first argument passed to next() treated differently?

```
function* calculate() {
  const a = yield 10;
  const b = yield a + 5;
  return b * 2;
}
const iterator = calculate();
console.log(iterator.next(100));
console.log(iterator.next(20));
console.log(iterator.next(7));
console.log(iterator.next());
```

> **Output:** {value: 10, done: false}, {value: 25, done: false}, {value: 14, done: true}, {value: undefined, done: true}. `next(100)` starts the generator; 100 is ignored. `yield 10` pauses. `next(20)` replaces `yield 10` with 20. `a = 20`, so yields 25. `next(7)` replaces `yield a+5` with 7. `b = 7`. Returns 14.

35. What is the output, and why does for...of behave differently from manual iteration here?

```
function* numbers() {
  yield 1;
  yield 2;
  return 3;
}
const iterator = numbers();
console.log([...iterator]);

const secondIterator = numbers();
console.log(secondIterator.next());
console.log(secondIterator.next());
console.log(secondIterator.next());
```

> **Output:** [1, 2]. Then {value: 1, done: false}, {value: 2, done: false}, {value: 3, done: true}. `for...of` and spread syntax `[...]` consume values until `done: true` and ignore the final `return` value. Manual iteration surfaces the return value.

36. What is the output, and why?

```
const array = [1, 2, 3];
for (const value of array) {
  console.log(value);
  if (value === 2) {
    array.push(4);
  }
}
```

> **Output: 1, 2, 3, 4.** `for...of` evaluates the iterator live against the array's state. Pushing an element dynamically extends the array's length, and the iterator will visit the newly added element.

37. What is the output, and why does the asynchronous execution order differ from what some developers expect?

```
async function processItems() {
  [1, 2, 3].forEach(async (value) => {
    await Promise.resolve();
    console.log(value);
  });
  console.log("Done");
}
processItems();
```

> **Output: Done, 1, 2, 3.** `forEach` does not await the inner async callbacks. The synchronous loop fires off three asynchronous tasks and immediately completes, logging "Done". The microtasks resolve afterward.

38. What is the output, and why?

```
const array = [1, , 3];
console.log([...array]);
console.log(array.map((value) => value * 2));
console.log(array.filter(() => true));

for (const value of array) {
  console.log(value);
}
```

> **Output:** Spread `[1, undefined, 3]`. Map `[2, empty, 6]`. Filter `[1, 3]`. for...of loop logs 1, undefined, 3. Array methods (map, filter) skip empty holes. Iterators (spread, for...of) visit them and treat them as `undefined`.

39. What is the output, and how does iterator cleanup work?

```
const iterable = {
  [Symbol.iterator]() {
    let value = 0;
    return {
      next() { return { value: value++, done: false }; },
      return() {
        console.log("Cleanup");
        return { done: true };
      }
    };
  }
};

for (const value of iterable) {
  console.log(value);
  if (value === 1) { break; }
}
```

> **Output: 0, 1, Cleanup.** When the `for...of` loop exits early due to the `break` statement, it automatically invokes the iterator's `return()` method if it exists. This mechanism is designed to allow proper cleanup of resources.

40. What is the output, and how does yield* propagate the delegated generator's return value?

```
function* child() {
  yield "A";
  return "B";
}

function* parent() {
  const result = yield* child();
  yield result;
}

console.log([...parent()]);
```

> **Output: ["A", "B"].** The `yield* child()` expression yields all values from the child generator (yielding "A"). Once the child generator finishes, its `return` value ("B") evaluates as the result of the `yield*` expression, which is stored in `result` and subsequently yielded by the parent.